JWT 인증을 적용한 나만의 게시판 만들기

수업에서 배운 게시판 CRUD와 JWT 인증을 연결하여, 누구나 게시글을 읽고 로그인한 사용자만 글을 작성하며 자신의 글만 수정·삭제할 수 있는 웹서비스를 완성하세요. 서비스의 주제는 자유롭게 정합니다. 구현 결과와 함께 핵심 코드 설명, 테스트 결과, AI 활용 및 검증 내역을 보고서로 제출합니다.

과제 운영

- 수행 방식 개인 과제. 프론트 1종과 백엔드 1종을 선택합니다.
- 실행 기준 본인 PC에서 프론트 빌드 결과와 백엔드·SQLite 파일 DB를 실행합니다. 교수자도 저장소를 clone해 로컬에서 재현할 수 있어야 합니다.
- 제출 방식 GitHub 저장소에 코드·README·보고서·테스트 증거를 올리고 저장소 URL을 LMS에 제출합니다.
- 제출 기한 LMS에 별도로 안내합니다.
- 주제 예시 영화 리뷰, 맛집 후기, 학습 기록, 취미 공유, 게임 공략.
- 제공 템플릿과 수업 예제를 수정해서 사용할 수 있습니다. 처음부터 구현해도 평가 기준은 같습니다.
- 프레임워크 선택, 디자인 화려함, 추가 기능 수에 따른 가산점은 없습니다. 기본 기능과 이해도를 중심으로 평가합니다.

기술 선택

구분	선택 가능한 기술	선택 방법
프론트엔드	React / Vue	둘 중 하나
백엔드	Node.js Express / Python FastAPI / Java Spring Boot	셋 중 하나
데이터 저장	SQLite 파일 DB 필수	세 백엔드 모두 data/board.sqlite 사용

총 6가지 조합 중 하나만 완성하면 됩니다. 공통 API 명세를 유지하세요. DB는 SQLite를 사용합니다. 코드 구조나 SQLite 파일 경로를 변경했다면 실행 방법과 변경 이유를 보고서에 설명하세요.

제공 범위와 구현 범위

제공하는 시작 코드에는 프로젝트 설정, 게시판 목록·상세·등록·수정·삭제, 파일 DB 저장, 기본 화면, 테스트 계정 두 개와 비밀번호 검증 함수가 포함되어 있습니다. 학생은 JWT 발급과 검증, 프론트 로그인 연결, 인증 오류 처리, 작성자 권한 검사를 완성합니다.

시작 코드의 인증 관련 TODO와 데모 사용자 반환은 의도적으로 미완성입니다. 로그인 API의 501 응답은 구현 전 상태를 의미합니다. 최종 제출본에서는 이 임시 동작을 제거해야 합니다.

시작과 작업 순서

실행 확인부터 구현 파일 찾기까지 진행한 뒤, 인증을 한 단계씩 완성하세요.

먼저 실행하고 구조를 확인하세요

1. 게시판 주제를 정하고 React/Vue 중 하나, Express/FastAPI/Spring 중 하나를 선택합니다. 여섯 조합을 모두 구현하는 과제가 아닙니다.
2. 선택한 백엔드 README대로 의존성을 설치하고 실행합니다. GET /api/health의 200과 JSON의 status: ok를 확인합니다.
3. 프론트를 실행해 목록·상세·등록·수정·삭제를 확인합니다. 처음에는 글이 없어도 정상이며, 직접 시험 글을 만들어 봅니다.
4. 시작 코드의 올바른 로그인은 501 NOT_IMPLEMENTED입니다. 인증을 완성하기 전의 의도된 상태입니다. 데모 사용자 ID 반환과 모든 수정 버튼 표시도 학생이 교체해야 합니다.
5. 요청 흐름을 추적합니다. 화면 이벤트 → 프론트 api.js → 백엔드 라우트 → 인증 함수 → 저장소 → JSON 응답 순서로 파일을 찾아보세요.

실제로 수정할 위치

선택 기술	핵심 수정 파일	학생 구현 항목
React	src/api.js, src/main.jsx	FE-01~04와 화면 상태
Vue	src/api.js, src/App.vue	FE-01~04와 화면 상태
Express	src/auth.js	BE-01~03
FastAPI	auth.py	BE-01~03
Spring Boot	AuthService.java	BE-01~03

Spring 파일은 src/main/java/edu/classroom/board/ 아래에 있습니다. 파일 경로는 선택한 기술 폴더 기준입니다. 라우트·설정·화면 등 다른 파일을 수정해도 되지만 변경 이유를 설명해야 합니다.

권장 구현 순서

서버 키 설정 → BE-01 로그인과 발급 → BE-02 검증과 /me → BE-03 작성자 권한 → FE-01~04 로그인 연결 → 오류 처리 → 빌드본 검증 → 보고서와 GitHub 제출 순서로 진행합니다. 한 단계씩 실제 요청으로 확인하고 다음 단계로 넘어가세요.

코드나 설명에 AI·수업 예제·외부 자료를 사용했다면 그때 파일과 함수, 수정 내용, 검증 결과를 기록합니다. 마지막에 기억만으로 사용 내역을 만들지 마세요.

백엔드 구현

로그인 성공부터 토큰 검증과 작성자 권한까지 서버에서 보장해야 합니다.

BE-01 로그인과 토큰 발급

1. 제공된 계정·비밀번호 검증 함수를 유지하고, 올바른 계정에만 토큰을 발급합니다. 임의의 이메일로 사용자를 만들거나 비밀번호 비교를 생략하지 않습니다.
2. JWT_SECRET을 새 난수로 생성해 서버에 설정합니다. 64자리 16진수를 32바이트 키로 변환하고, 빈 값·형식 오류·제공 예시 키는 거부합니다. Spring은 .env를 자동으로 읽지 않습니다.
3. issueToken 또는 issue_token의 임시 501을 실제 JWT 발급으로 교체합니다. HS256, 문자열 sub, exp, iss, aud와 설정한 TTL을 사용합니다. 비밀번호와 해시는 토큰에 넣지 않습니다.
4. 로그인 성공은 200이며 accessToken, expiresIn, user를 반환합니다. user는 id·name·email입니다. 틀린 비밀번호와 없는 계정은 모두 401 INVALID_CREDENTIALS, 입력 형식 오류는 400 BAD_INPUT입니다.

BE-02 검증된 사용자 확인

1. currentUserId 또는 current_user_id의 고정 ID 반환을 제거합니다. 보호 API는 요청마다 Authorization 헤더를 확인해야 합니다.
2. 토큰이 없거나 Bearer 헤더 형식이 틀리면 401 TOKEN_REQUIRED로 처리합니다. Bearer 뒤에 토큰처럼 전달된 값이 JWT로 유효하지 않으면 401 INVALID_TOKEN입니다.
3. 라이브러리로 허용 알고리즘·서명·exp 존재와 만료·iss·aud·sub를 검증합니다. 단순 decode 결과만 믿지 않습니다. 정상 서명의 만료 토큰은 401 TOKEN_EXPIRED, 변조·클레임 오류는 401 INVALID_TOKEN입니다.
4. sub를 사용자 ID로 해석하고 실제 계정이 존재하는지도 확인합니다. 검증된 ID만 /me와 등록·수정·삭제 처리에 전달합니다. 토큰의 사용자나 요청 본문을 그대로 신뢰하지 않습니다.
5. 보호 API의 401에는 WWW-Authenticate: Bearer 헤더를 포함합니다. /me 성공은 200이며 현재 계정의 공개 정보만 반환합니다.

BE-03 작성자 권한

등록할 때 authorId는 검증된 사용자 ID로 서버가 결정합니다. 수정·삭제는 저장된 글의 authorId와 현재 사용자 ID를 비교합니다. 다른 작성자이면 403 FORBIDDEN으로 거부하고 DB를 바꾸지 않습니다. 수정 성공 시 기존 작성자와 createdAt을 유지합니다. 프론트 버튼을 숨겨도 이 서버 검사는 반드시 남아 있어야 합니다.

프론트와 데이터 처리

API 응답을 화면 상태와 연결하고, 실패한 요청은 성공으로 표시하지 마세요.

FE-01 로그인 상태 연결

로그인 폼의 email·password를 JSON으로 POST /api/auth/login에 보냅니다. 성공 응답의 토큰과 user를 메모리에 저장하고 로그인 사용자 이름을 화면에 표시합니다. 실패 시 오류를 표시하고 성공 화면으로 이동하지 않습니다. 토큰은 localStorage·sessionStorage에 저장하지 않습니다.

FE-02 요청에 토큰 첨부

authorizationHeaders에서 저장한 토큰을 Authorization: Bearer <JWT> 형태로 보냅니다. 로그인 상태에서 /me와 글 등록·수정·삭제 요청에 실제 헤더가 들어가는지 Network에서 확인합니다. 토큰이 없으면 Bearer undefined 같은 헤더를 만들지 않습니다. 공개 목록과 상세는 토큰 없이도 조회되어야 합니다.

FE-03 응답 코드별 화면 처리

- 200 / 201: 반환된 결과로 목록·상세·사용자 상태를 갱신합니다. 서버가 실패했는데 성공 안내를 표시하지 않습니다.
- 204: 삭제가 성공한 것입니다. JSON을 파싱하지 않고 목록과 선택 상태를 갱신합니다.
- 400: 입력 오류를 알리고 수정할 수 있게 합니다. 입력값을 무조건 지우지 않습니다.
- 401: 메모리의 토큰·사용자와 화면 로그인 상태를 해제합니다. 로그인 실패, 토큰 없음, 변조, 만료를 code에 맞게 안내합니다.
- 403: 권한 부족을 알립니다. 토큰 자체가 무효한 상황이 아니므로 자동 로그아웃시키지 않습니다.
- 404: 해당 글이 없음을 알리고 목록으로 돌아가거나 목록을 갱신합니다.

FE-04 작성자별 버튼과 로그아웃

canEdit는 로그인 사용자 ID와 글의 authorId를 비교합니다. 비로그인 사용자에게는 글 작성 시 로그인 안내를 표시하고, 타인의 수정·삭제 버튼은 숨기거나 비활성화합니다. 로그아웃은 메모리 상태와 화면 표시를 즉시 지웁니다. 새로고침 후 로그인 해제는 정상입니다. 인증 미완성 데모 배너는 최종 서비스 안내로 바꿉니다.

입력과 데이터 유지

서버에서 title·content의 필수 여부, 문자열 형식, 공백, 길이를 확인합니다. 등록·수정에는 두 필드만 받으며 authorId·userId·id 같은 추가 필드는 400 BAD_INPUT입니다. 화면 검사만으로 끝내지 마세요. 실패한 요청 이후 공개 조회로 글이 바뀌지 않았는지 확인합니다. 같은 폴더에서 백엔드를 종료·재시작해 글·작성자·내용이 유지되는지도 확인합니다.

HTTP 응답 코드

상태 숫자와 JSON code, 응답 헤더 및 본문을 함께 확인합니다.

HTTP 상태 코드는 필수 구현 기준입니다. 본문에 숫자만 적거나 모든 요청을 200으로 반환하면 요구사항을 충족하지 않습니다. 아래 HTTP는 실제 응답 상태이며, 오류 code는 JSON 본문의 문자열입니다.

정상 응답

요청	HTTP	반환 내용
GET /api/health	200	status가 ok인 상태 객체
POST /api/auth/login	200	accessToken, expiresIn, user
GET /api/me	200	id, name, email
GET /api/posts	200	게시글 배열, 없으면 []
GET /api/posts/{id}	200	해당 게시글 객체
POST /api/posts	201	생성된 글과 Location 헤더
PUT /api/posts/{id}	200	수정된 글 객체
DELETE /api/posts/{id}	204	응답 본문 없음

실패 응답

HTTP	JSON의 code	발생 조건
400	BAD_INPUT	필수값·형식·길이·추가 필드·ID 오류
401	INVALID_CREDENTIALS	로그인 계정 또는 비밀번호 불일치
401	TOKEN_REQUIRED	보호 API의 토큰 누락·Bearer 형식 오류
401	INVALID_TOKEN	변조·서명·클레임 오류·없는 사용자
401	TOKEN_EXPIRED	정상 서명의 토큰이 만료됨
403	FORBIDDEN	인증한 사용자가 다른 사람 글을 변경
404	NOT_FOUND	유효한 ID의 글이 존재하지 않음

201은 새 글 생성에 사용하고 Location: /api/posts/{id}를 보냅니다. 204는 JSON 객체나 null도 보내지 않습니다. 보호 API의 401에는 WWW-Authenticate: Bearer를 보냅니다. 모든 실패 본문은 code와 message를 포함하며 message 문구는 자유입니다.

필수 기능과 JWT 기준

구현을 마친 뒤 아래 요구사항을 하나씩 최종 점검하세요.

1. 공개 조회 비로그인 상태에서 게시글 목록과 상세 내용을 볼 수 있어야 합니다.
2. 로그인 올바른 계정 정보에만 JWT를 발급하고 화면에 로그인 사용자를 표시해야 합니다. 잘못된 계정 정보에는 토큰을 발급하지 않습니다.
3. 내 정보 조회 검증된 JWT의 사용자 ID로 본인의 공개 정보를 반환합니다. 비밀번호나 해시는 응답에 포함하지 않습니다.
4. 글 작성 유효한 JWT가 있는 사용자만 작성할 수 있습니다. 작성자 ID는 서버가 토큰을 검증하여 결정합니다.
5. 수정과 삭제 작성자 본인만 가능합니다. 프론트 버튼 표시와 별개로 서버에서도 검사해야 합니다.
6. 예외 처리 토큰 없음·변조·만료는 401, 인증된 다른 작성자의 수정·삭제는 403, 존재하지 않는 글은 404로 처리합니다.
7. 입력 검사 제목 1~100자, 내용 1~5000자이며 공백만 입력할 수 없습니다. 서버에서 검사하고 추가 작성자 필드를 거부합니다.
8. 로그아웃과 만료 로그아웃 시 이 탭의 토큰과 사용자 상태를 지웁니다. 401 응답을 받으면 로그인 상태를 해제하고 재로그인을 안내합니다.
9. 데이터 유지 페이지 새로고침과 서버 재시작 후에도 게시글이 유지되어야 합니다.

이번 과제의 토큰은 브라우저 메모리에 보관합니다. 새로고침하면 로그인 상태가 해제되는 것이 정상입니다. 게시글 데이터 유지와 로그인 상태 유지를 구분하세요. 로그아웃은 이미 복사된 JWT를 서버에서 즉시 무효화하는 기능이 아닙니다.

회원가입, 비밀번호 재설정, Refresh Token, 소셜 로그인, 관리자 기능, 이미지 업로드, 공개 배포는 필수 범위에 포함하지 않습니다. 테스트 계정 Alice와 Bob을 이용합니다.

JWT 구현 기준

HS256과 서버 비밀 키를 사용합니다. 토큰에는 문자열 사용자 ID인 sub, 만료 시각 exp, 발급자 iss, 사용 대상 aud를 포함합니다. 허용 알고리즘을 고정하고 서명·만료·발급자·사용 대상과 필수 클레임을 검증합니다. JWT 내용을 읽기만 하는 decode는 검증을 대신하지 않습니다. 기본 유효 시간은 90초이며, 테스트용으로 짧게 변경할 수 있어야 합니다.

비밀 키는 서버 환경설정으로 주입하고 프론트 코드나 보고서에 포함하지 않습니다. 암호화·서명 알고리즘을 직접 만들지 말고 제공된 JWT 라이브러리를 사용하세요. 구체적인 요청과 응답은 공통 API 명세를 따릅니다.

검증 순서와 응답 판단

401과 403을 구분하고, 여러 조건이 겹친 요청도 직접 시험하세요.

헷갈리기 쉬운 응답 구분

401은 인증 실패, 403은 인증 후 권한 부족, 404는 글 없음입니다. 유효한 ID와 정상 JSON으로 보호 요청을 보낼 때는 인증 → 글 존재 → 작성자 권한 → 입력 검증 → DB 변경 순서로 판단합니다. ID 형식 오류는 400으로 처리하며, JSON 자체가 깨진 요청은 프레임워크가 먼저 400으로 거부할 수 있습니다.

- 토큰 없이 없는 글을 수정: 401 TOKEN_REQUIRED. 정상 토큰으로 같은 글을 수정: 404 NOT_FOUND.
- Bob의 정상 토큰으로 Alice 글을 수정: 403 FORBIDDEN. Bob의 토큰이 만료되었다면 401 TOKEN_EXPIRED.
- Alice가 자기 글에 공백 제목을 제출: 400 BAD_INPUT. Bob이 같은 글에 공백 제목을 제출: 403 FORBIDDEN.
- GET /api/posts/abc: 400 BAD_INPUT. 정상 양의 정수 ID지만 실제 글이 없으면 404 NOT_FOUND.

FastAPI의 기본 422, Spring의 로그인 페이지 이동, HTML 오류 페이지가 지정 API에 그대로 나오지 않도록 공통 JSON 응답에 맞춥니다. 예상 가능한 입력·인증·권한 오류를 500으로 처리하지 않습니다. 501 NOT_IMPLEMENTED는 시작 코드 전용이며 최종 로그인 성공에서 남아 있으면 미완성입니다.

실제 요청으로 확인하는 방법

1. 브라우저 개발자 도구의 Network에서 요청 메서드·URL, Status Code, Request Headers, Response를 확인합니다. 화면 알림만으로 HTTP 코드를 확인했다고 기록하지 않습니다.
2. 먼저 Alice로 로그인하고 글을 생성합니다. Bob으로 로그인한 뒤 API 클라이언트나 curl로 Alice 글의 수정·삭제를 직접 요청합니다. 숨겨진 버튼을 클릭하지 못했다는 사실만으로 권한 검사를 통과한 것이 아닙니다.
3. 토큰 없는 요청, 서명 부분을 바꾼 토큰, 짧은 TTL로 실제 만료시킨 토큰을 각각 시험합니다. 만료 검사는 새 토큰을 발급받고 expiresIn보다 충분히 기다립니다.
4. T01~T17을 수행하고 요청 조건·예상 상태·실제 상태·JSON code·화면 처리·데이터 유지 여부·증거 위치를 기록합니다. 실패한 변경 요청은 재조회하여 원문이 그대로인지 확인합니다.
5. 자동 검사와 별개로 빌드된 화면, 로그아웃·새로고침과 실제 서버 재시작을 확인합니다. 미실행 항목은 미실행으로 적습니다.

자동 API 검사: python3 tools/check_api.py --mode final

만료 검사: TOKEN_TTL=3으로 서버를 재시작한 뒤 python3 tools/check_api.py --mode final --expiry

Windows는 python3 대신 py를 사용할 수 있습니다. 검사 도구는 일부 항목만 자동 확인하므로 전체 수동 시나리오를 대체하지 않습니다.

제출과 로컬 실행

GitHub 저장소를 clone한 뒤 README만으로 실행할 수 있어야 합니다.

제출물과 GitHub 제출 방법

- 소스 코드 선택한 프론트와 백엔드 폴더, 의존성 파일과 lock 파일, 필요한 설정 예시.
- 루트 README.md 서비스 소개, 작성자, 기술 조합과 버전, 폴더 구조, 설치·실행 명령, 접속 주소, 테스트 계정, 환경설정과 DB 초기화 방법, 구현 완료 여부를 작성합니다. 보고서·테스트 증거 링크와 AI 활용 요약도 포함합니다.
- 보고서 PDF 제공 양식을 바탕으로 6~10쪽 내외를 권장합니다. 분량 자체는 평가하지 않으며 필요한 설명과 증거를 담으세요.
- 테스트 증거 보고서에 요청·상태 코드·화면 캡처를 삽입하고, 필요한 경우 별도 evidence 폴더에 첨부합니다.

GitHub 저장소에는 선택한 frontend/, backend/, 루트 README.md, docs/report.pdf, docs/evidence/를 올리세요. 폴더 이름을 다르게 사용해도 README의 경로와 명령이 일치하면 됩니다. 제공한 06_제출용_README.md를 루트 README.md로 복사해 작성할 수 있습니다.

저장소 URL과 제출 기준 커밋 SHA를 LMS에 제출하세요. 공개 저장소 또는 교수자가 열람·복제할 수 있는 비공개 저장소를 사용합니다. 비공개 저장소는 수업에서 안내한 교수자 GitHub 계정에 접근 권한을 부여하고 확인하세요. 새 폴더에 clone한 뒤 README만으로 실행되는지, GitHub에서 보고서와 증거 링크가 열리는지 확인하세요.

미사용 템플릿과 node_modules, .venv, target, 실제 .env, DB 파일, 비밀 키, 실제 토큰은 커밋하지 않습니다. API 증거의 토큰은 일부만 남기고 가립니다. ZIP 제출과 별도 시연 영상은 요구하지 않습니다. 보고서 PDF도 같은 저장소에 포함하며 README에서 연결합니다.

로컬 배포와 실행 기준

GitHub는 소스와 문서 제출용입니다. 별도 서버·도메인·클라우드 배포 주소는 요구하지 않습니다. 선택한 백엔드를 실행한 뒤 프론트 폴더에서 npm ci → npm run build → npm run preview 순서로 실행합니다. 브라우저 접속 주소는 http://127.0.0.1:4173, 백엔드는 http://127.0.0.1:8000입니다. 개발 중에는 npm run dev와 5173번 주소를 사용할 수 있습니다.

README에는 실제 폴더 기준 설치·환경설정·빌드·실행·종료 명령을 적으세요. 새 폴더에서 설치하고 빌드된 화면의 API 연결과 데이터 재시작 유지를 확인한 증거를 보고서에 포함합니다. 운영체제별 절차는 07_로컬실행안내.md를 참고하세요.

보고서 작성 기준

서비스 목적과 기술 선택 이유, 전체 구조, 실제로 수정한 파일 목록을 먼저 설명하세요. 핵심 코드 네 부분인 로그인·발급, 토큰 전송, 서버 검증, 작성자 권한 검사를 각각 파일 경로와 함수명으로 연결해 설명합니다. 코드는 필요한 부분만 발췌하고 입력, 처리, 출력, 실패 조건을 자신의 말로 작성하세요.

테스트는 성공 화면만 제시하지 말고 토큰 없음·변조·만료, 다른 사용자의 수정·삭제 시도와 서버 응답을 포함하세요. 확인하지 못한 항목은 미확인으로 표시하고 알려진 문제를 솔직하게 적으세요.

AI 활용과 평가

AI 활용 여부는 명확히 기록하고, 제출 코드와 검증 결과를 자신의 말로 설명하세요.

AI 활용 및 출처 표기

AI는 개념 학습, 설계, 코드 생성, 디버깅, 테스트, 문장 정리에 사용할 수 있습니다. 사용 여부나 사용량 자체로 가감점하지 않습니다. 사용했다면 도구·모델(확인 가능한 경우), 사용 목적, 주요 요청, 반영한 파일·함수, 직접 수정한 내용과 검증 결과를 기록하세요. 사용하지 않았다면 미사용이라고 명시하세요.

대표 활용 사례 2~3건을 자세히 기록하되, 실제 사용이 1건이면 그 1건만 작성합니다. 다른 사용 내역은 요약 목록으로 적습니다. AI가 생성한 코드를 그대로 반영한 경우도 표시하고, 왜 동작한다고 판단했는지 증거를 제시하세요. 전체 대화 내역 공개는 요구하지 않습니다.

수업 코드, 외부 코드, AI 도움, 본인 작성 부분을 구분하세요. 외부 자료의 URL과 사용 위치를 적고 필요한 라이선스·저작권 표시를 유지하세요. 보고서에 본인이 수행하지 않은 테스트나 수정 과정을 만들어 쓰지 마세요. 제출자는 모든 제출 코드의 동작을 설명할 수 있어야 합니다.

평가 기준

평가 항목	배점	확인하는 내용
CRUD와 데이터 유지	20	목록·상세·등록·수정·삭제, DB 저장과 재시작 유지
JWT 인증	25	올바른 발급·검증, 보호 API 적용, 사용자 확인
작성자 권한	20	다른 사용자의 수정·삭제 거부, 작성자 위조 방지
화면과 오류 처리	10	로그인 사용자 표시, 토큰 전송, 만료·로그아웃 안내
코드 설명과 설계 이해	10	실제 파일과 설명의 일치, 인증과 권한의 구분
테스트와 재현성	10	실패 사례 증거, 로컬 빌드·실행 재현, 결과의 신뢰성
AI 활용과 출처 기록	5	사용 또는 미사용의 명확한 표기와 검증·출처 설명
합계	100	기술 조합과 무관하게 동일 적용

제공 기능도 제출본에서 계속 동작해야 합니다. AI 미사용 학생도 미사용 표기와 활용 자료의 출처를 명확히 기록하면 해당 항목의 만점을 받을 수 있습니다. 추가 기능으로 필수 인증·권한 요구사항을 대체할 수 없습니다.